Database Systems: Lecture Notes (Lectures 1-9)

Table of Contents

Database Systems: Complete Lecture Notes (Lectures 1 to 9) 1

Lecture 1: Introduction to Database, DBMS, Data and Information 1

Lecture 2: File Processing Systems vs Database Management Systems 3

Lecture 3: Data Flow Diagrams (DFDs) 6

Lecture 4: DFD Example, Schema and Subschemas, Data Independence 8

Lecture 5: Data Independence and Evolution of DB Systems 10

Lecture 6: Hierarchical and Network Models 12

Lecture 7: Object-Oriented and Relational Models 14

Lecture 8: Functions of DBMS, DBMS Architecture and Metadata 16

Lecture 9: Relational Data Model, Tables, Keys and Constraints 18

Final Revision Sheet 22

Lecture 1: Introduction to Database, DBMS, Data and Information

Course aim

- Learn database modeling and development.
- Learn SQL and database implementation in MS SQL Server.
- Reference books: Modern Database Management (Hoffer, Venkataraman, Topi) and Database Systems: A Practical Approach (Connolly and Begg).

Data vs Information

- **Data** is raw, unprocessed facts. Example: 45, 67, 82, 90.
- **Information** is data that has been processed and given meaning. Example: “The class average is 71.”
- A database stores data, and queries and reports turn it into information.

What is a database?

The slides define it as: “A **shared** collection of **logically related** data, and a **description** of this data, designed to meet the **information needs of an organization**.”

Each key word means something: - **Shared:** many users and departments use the same data. Payroll, inventory and vendors all draw from one place. - **Logically related:** the data belongs together. A hospital database holds patients, doctors and appointments, not random facts. - **Description of the data (metadata):** the database also stores what each field means, its type and its size. - **Information needs of the organization:** it exists to support the organization’s decisions and operations.

The database is also called a **data repository**, meaning a central store.

Real-life analogy: A university’s student records office is a central cabinet. Accounts, exams and hostel departments all consult the same cabinet instead of keeping their own copies. A “management system” sits at the door and controls who can open which drawer. That door-keeper is the DBMS.

Logically related data has three parts

- **Entities:** things we store data about (Student, Course).
- **Attributes:** properties of those things (Name, Roll No).
- **Relationships:** links between entities (a Student enrolls in a Course).

System catalog (metadata)

- The catalog describes the data and gives **program-data independence**. Programs do not need to know the storage details.
- **Metadata is data about data.**

What is a DBMS?

A **Database Management System** is software that lets users **define, create, maintain and control access** to the database.

Mnemonic: D-C-M-C (Define, Create, Maintain, Control access).

Example DBMS: property rental company

The slide shows Sales and Contracts staff using application programs. Those programs talk to **one DBMS**, which talks to **one database**. The database holds these tables: - PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo) - PrivateOwner (ownerNo, fName, lName, address, telNo) - Client (clientNo, fName, lName, address, telNo, prefType, maxRent) - Lease (leaseNo, propertyNo, clientNo, paymentMethod, deposit, paid, rentStart, rentFinish)

Notice that Lease contains propertyNo and clientNo. That is how tables link to each other, which becomes important in Lecture 9 (foreign keys).

Components of a DBMS

1. **DDL (Data Definition Language):** specifies data types, structures and constraints. It builds the "shape" of the data. Example: CREATE TABLE.
2. **DML (Data Manipulation Language):** the query facility for asking questions of the data. Example: SELECT, INSERT.
3. **Controlled access**, which includes four systems:
   - **Security system:** who may see what.
   - **Integrity system:** keeps data correct and consistent.
   - **Concurrency control system:** handles many users at the same time.
   - **Recovery control system:** restores data after a crash.

Analogy: A bank. The DDL is designing the account forms. The DML is depositing and withdrawing. Security is the vault guard. Integrity is making sure balances add up. Concurrency is two people withdrawing from the same joint account at once. Recovery is restoring records after a power cut.

DBMS big picture (slide 12)

Applications, other DBMSs and users all connect to the DBMS. It defines, records, queries, updates and manages data in the storage area. Storage can hold relational, hierarchical, flat-file or object databases.

Lecture 2: File Processing Systems vs Database Management Systems

Traditional file-based system

- **Definition:** a collection of application programs that perform services for end users, such as producing reports. **Each program defines and manages its own data.**
- Slogan: **One file, one application.**

Real-life example: Stereos To Go, a shop, has three programs: - An Invoice Program with its own Customer Orders file. - A Customer Account Program with its own Customer Accounts file. - A Customer Mailing Program with its own Mailing List file.

All three files store the customer's name and address separately.

Limitations of file-based systems (very important)

1. **Separation and isolation of data:** each program only sees its own file.
2. **Duplication of data (redundancy):** the same customer info is stored in several places.
3. **Loss of data integrity:** if the customer moves, one file is updated and the others are not. Which address is correct? There is no consistency.
4. **Data dependence:** the program defines the data, so changing a file's structure means rewriting the program.
5. **Incompatibility of file formats:** different programs and languages store data differently.
6. **Fixed queries:** only the reports programmers wrote are available. There is little flexibility when information needs change.

Mnemonic: S-D-I-D-I-F (Separation, Duplication, Integrity loss, Dependence, Incompatibility, Fixed queries).

Data redundancy example

The Customer Order file, Customer Account file and Mailing List file all store the customer's name, address, city, state and zip. That is one fact stored three times. If the address changes, all three must be updated, and one will eventually be missed.

Data abstraction

- This means separating the **structure (definition) of data** from the **application programs**.
- In older COBOL systems, the data layout was written inside the program, which tightly binds the data file and the program.
- In the database approach, the definitions live in the central repository with the data. Applications (on client or server) access data through the DBMS.

Analogy: When you drive a car, you use the steering wheel and pedals. You do not need to know how the engine works. If the mechanic changes the engine, your driving does not change. That is abstraction.

Organizing data

- **Entity:** a distinct object (person, place, thing, concept or event). Example: Customers.
- **Attribute:** describes an aspect of an entity. Example: Account_number, Name, Address.
- **Relationship:** an association between entities. Example: Customers make Purchases (Purchases has Invoice_number, Account_number, Purchase_date).

Stereos To Go database

Customer Orders, Order Items, Products and Manufacturers all sit in one **central repository (an organizational resource)**. The DBMS gives a **single access point** to a **multitude of applications**: management queries, application programs and other software. The DBMS provides DDL, DML and controlled access.

Components of a database environment (HW-SW-D-P-P)

1. **Hardware:** the machines.
2. **Software:** DBMS, application programs and query software.
3. **Data:** organized in a **schema** and partitioned into **subschemas**.
4. **Procedures:** rules governing design, access and use.
5. **People:**
    - Administrators: DA (Data Administrator) and DBA (Database Administrator).
    - Designers: logical and physical.
    - Application developers.

- Users: novice and high-powered.

Advantages of the database approach

1. Control of data redundancy.
2. Data consistency.
3. Greater informational gain: more information from the same amount of data.
4. Data sharing: data becomes a shared organizational resource.
5. Improved data integrity, validity and consistency.
6. Improved access and security.
7. Enforcement of standards.
8. Economy of scale through centralization and consolidation.
9. Balancing conflicting requirements: the DBA oversees data and definitions.
10. Improved data accessibility and responsiveness.
11. Increased productivity.
12. Improved maintenance through data independence.
13. Increased concurrency.
14. Improved backup and recovery services.

Disadvantages of the database approach

1. **Complexity.**
2. **Size:** it needs dedicated technology and people.
3. **Cost of DBMS.**
4. **Additional hardware costs.**
5. **Cost of conversion** from old systems.
6. **Performance:** in a production environment, processing can be slow.
7. **Higher impact of failure:** since everything is centralized, one failure hurts everyone.

Memory trick: Everything centralized means one big win and one big risk. A single shared kitchen is efficient, but if it burns down, the whole hostel starves.

Lecture 3: Data Flow Diagrams (DFDs)

This lecture repeats the entity/attribute/relationship, components and advantages/disadvantages slides, then moves on to DFDs.

What is a DFD?

A **Data Flow Diagram** is a picture of how data moves through a system: where it comes from, what processes handle it, where it is stored and where it goes.

Steps to create a DFD

1. Create a **list of activities**.
2. Construct the **Context Level DFD**. It identifies external entities (sources and sinks) and shows the whole system as one process.
3. Construct the **Level 0 DFD**. It identifies manageable sub-processes.
4. Construct **Level 1 to n DFDs**. They identify the actual data flows and data stores.
5. **Check against the rules of DFD.**

The slides note an “old way” (no use-case diagram) and a “new way” (use a use-case diagram to get the list of activities).

Naming guidelines (very examinable)

- **External entity:** a noun (Customer, Vendor).
- **Data flow:** names of data (Order, Payment).
- **Process:** a verb phrase (Record Order), or a system or subsystem name.
- **Data store:** a noun (Inventory, Time Cards).

Mnemonic: Entity = Noun, Flow = Name of data, Process = Verb, Store = Noun.

Lemonade stand example (the whole process)

Step 1: Activities. Customer Order, Serve Product, Collect Payment, Produce Product, Store Product. Support activities: Order Raw Materials, Pay for Raw Materials, Pay for Labor.

These are then grouped logically into four areas: - **Sale:** Customer Order, Serve Product, Collect Payment. - **Production:** Produce Product, Store Product. - **Procurement:** Order Raw Materials, Pay for Raw Materials. - **Payroll:** Pay for Labor.

Step 2: Context level (0.0 Lemonade System). - External entities: **Customer**, **Employee**, **Vendor**. - Customer sends Order and Payment, and receives Product Served. - Vendor receives Purchase Order and Payment, and sends Received Goods. - Employee receives Pay and Production Schedule, and sends Time Worked. - A Sales Forecast also leaves the system.

Step 3: Level 0. The system splits into four processes: **1.0 Sale, 2.0 Production, 3.0 Procurement, 4.0 Payroll.**

Step 4: Level 1. Each process is broken down and data stores appear: - 1.0 Sale becomes 1.1 Record Order, 1.2 Receive Payment and 1.3 Produce Sales Forecast. Stores: ORDER, PAYMENT. - 2.0 Production becomes 2.1 Serve Product, 2.2 Produce Product and 2.3 Store Product. Stores: ORDER, RAW MATERIALS, INVENTORY. - 3.0 Procurement becomes 3.1 Produce Purchase Order, 3.2 Receive Items and 3.3 Pay Vendor. Stores: PURCHASE ORDER, RAW MATERIALS, RECEIVED ITEMS. - 4.0 Payroll becomes 4.1 Record Time Worked, 4.2 Calculate Payroll and 4.3 Pay Employee. Stores: TIME CARDS, PAYROLL, PAYMENTS.

Process decomposition

The numbering shows the hierarchy: - Context: 0.0 - Level 0: 1.0, 2.0, 3.0, 4.0 - Level 1: 1.1, 1.2, 2.1, …

Analogy: Think of Google Maps. The context level is the country view, Level 0 is the city view, and Level 1 is the street view.

Lecture 4: DFD Example, Schema and Subschemas, Data Independence

DFD example: bus garage repairs

Story: Buses come to a garage. A mechanic and helper repair them, record the reason and the total parts cost on a **Shop Repair Order**. The information is used for: - Billing by Accounting. - Parts monitoring by the Inventory Management System. - Performance review by the Supervisor.

Identified elements: - **External entities:** Bus, Mechanic, Helper, Supervisor, Inventory Management System, Accounting Department. - **Key process (the system):** performing repairs and storing repair information. - **Processes:** record Bus ID and reason, determine parts needed,

perform repair, calculate parts extended and total cost, record labor hours and cost. - **Data stores:** Personnel file, Repairs file, Bus master list, Parts list. - **Data flows:** Repair order, Bus record, Parts record, Employee timecard, Invoices.

Context diagram flows: Bus sends “mechanical problem to be repaired” and receives “fixed mechanical problems”. Mechanic and Helper provide labor. The system sends a repair summary to the Supervisor, a list of parts used to Inventory, and labor and parts cost details to Accounting.

The slide ends with the exercise: **Prepare the 0-level and 1-level diagrams** (practice this).

Schema and subschemas

- **Schema:** the complete catalog (description) of all data in the database.
- **Subschema:** an individual **view** of the database for one group of users.
- The DBMS software manages the database. Below it is the physical database.

Analogy: A university database holds everything. The exam department sees only marks and roll numbers. The accounts office sees only fees. The hostel office sees only room allocation. Each gets its own subschema, a slice of the whole.

Three-level architecture (ANSI-SPARC)

Level	Schema	Describes	Viewpoint
External	External schemas (subschemas)	The part of the database relevant to one user	User’s view
Conceptual	Conceptual schema	**What** data is stored and the relationships	Community view
Internal	Internal schema	**How** data is stored	Physical representation

Below the internal level is the physical data organization (physical storage).

Mnemonic: External = what I see, Conceptual = what exists, Internal = how it is stored.

Analogy (restaurant): - External level: the menu as a customer sees it. - Conceptual level: the full list of dishes and ingredients the restaurant offers. - Internal level: how ingredients are physically stored in fridges and shelves.

Why this architecture? (Independence goals)

- Each user accesses the same data but with a **customized view**.
- Users do not deal with physical storage details.
- The DBA can change storage structures **without affecting user views**.
- The internal structure is unaffected by changes to the physical aspects of storage.
- The DBA can change the conceptual structure **without affecting all users**.

Lecture 5: Data Independence and Evolution of DB Systems

Data independence (two types)

1. **Logical data independence:** the **immunity of external schemas to changes in the conceptual schema.** Example: you add a “Mobile Number” column to the Student table, and the exam department’s report program continues working unchanged.
2. **Physical data independence:** the **immunity of the conceptual schema to changes in the internal schema.** Example: the DBA moves data to a faster SSD or changes the storage structure. Tables and queries do not change.

Slide tagline: **“Plug and Play!”** You can swap one layer without breaking the others.

Analogy: You can change your phone’s battery (physical) or install a new app (logical), and your contacts (the conceptual data) stay intact.

Database environment vs file-based systems

- **Database environment:** users, via subschemas, go through one schema and DBMS to a **shared and managed** physical database.
- **File-based system:** each user or program has its own file. There is no DBMS between them and the data, so **integrity problems** arise between the copies. In the slide’s diagram, one file is open to everyone, so everyone has access to all the data in it. That is a security problem too.

Database languages: DDL vs DML

- **DDL (Data Definition Language):** names the entities required by the application and the relationships between them. It **specifies or modifies the schema and subschemas**. Commands: CREATE, ALTER, DROP.

- **DML (Data Manipulation Language):** supports basic data operations. It **reads and updates** the database (insert, update, delete, and read or query). Commands: SELECT, INSERT, UPDATE, DELETE.

Memory: DDL builds the house. DML lives in the house.

Evolution of database systems (timeline)

- Flat files: 1960s to 1980s
- Hierarchical: 1970s to 1990s
- Network: 1970s to 1990s
- Relational: 1980s to present
- Object-oriented: 1990s to present
- Object-relational: 1990s to present
- Data warehousing: 1980s to present
- Web-enabled: 1990s to present

(Lecture 6's version of the chart adds **NoSQL, JSON, XML around 2015**.)

Mnemonic for order: File, **H**ierarchical, **N**etwork, **R**elational, **O**bject, **O**bject-relational, then NoSQL.

Models

A **model** represents the real thing, identifies the components and their interactions, and specifies behavior. The slide's example is a model aeroplane: it is not the real plane but captures how it is built.

Data model

An integrated collection of concepts for describing and manipulating data, relationships between data and constraints on data. It has three components: 1. **Structural part:** rules for constructing the database. 2. **Manipulative part:** the operations allowed on the data. 3. **Integrity rules:** ensure the accuracy of the data.

Mnemonic: S-M-I (Structure, Manipulation, Integrity).

Database models: conceptual vs implementation

- **Conceptual models:** emphasize **what** is represented (the logical nature of the data). They are used as a **blueprint** for design. Examples: Entity-Relationship (ER) model and the Object-oriented model.
- **Implementation models:** emphasize **how** data is represented in the database. Examples: Hierarchical, Network, Relational, Object-oriented.

Analogy: A conceptual model is the architect's blueprint. An implementation model is the actual construction method (bricks, concrete or steel frame).

Relationship types in conceptual models

- **One-to-one (1:1):** one person has one passport.
- **One-to-many (1:M):** one teacher teaches many students.
- **Many-to-many (M:N):** students and courses (many students take many courses, and each course has many students).

Lecture 6: Hierarchical and Network Models

Hierarchical Database Model (HDBM)

Concept: Data is organized like an **upside-down tree**. - Each **parent** can have many **children**. - Each **child has only one parent.** - One table is the **root**, and other tables **branch** out from it. - Relationships are parent-child, connected by **pointers** (links). - It represents **1:M** relationships only.

Example (slide): A filing cabinet manufacturer. - Root: Final assembly (filing cabinet) - Level 1: Component A, Component B, Component C - Level 2: Assembly A (under A), Assembly B and Assembly C (under C) - Level 3: Parts A, B (under Assembly A) and Parts C, D, E (under Assembly B)

Real-life analogy: A family tree or a company org chart. Each employee reports to exactly one manager.

Hierarchical path (preorder traversal / left-list sequence): you read the tree starting from the left, going down before going across. In the slide's example: Final assembly,

Component A, Assembly A, Part A, Part B, Component B, Component C, Assembly B, Part C, Part D.

If a segment is frequently accessed, it can be **re-listed** in the sequence.

Banking example: One customer account is subject to many transactions (1:M). The relationship is fixed (debiting and crediting), and there are large numbers of transactions. This is why bank systems commonly used HDBM.

Advantages: - Conceptual simplicity (the relationship between layers is simple). - Database security enforced uniformly. - Data integrity. - Data independence. - Efficient for 1:M relationships and large numbers of transactions. - Dominant in the **1970s** on mainframes with large databases.

Disadvantages: - Complex implementation (physical storage characteristics, complicated design). - Difficult to manage and a lack of standards. - **Lacks structural independence.** - Application programming and use are complex (pointer-based). - Implementation limitations: it **only handles 1:M**.

Network Database Model (NDBM)

Concept: Each record can have multiple parents. - It was standardized by the **DBTG (Database Task Group)**. - It introduces the **set** to describe relationships. - Each set has an **owner record** and a **member record** (parallel to parent and child in HDBM). - A member can have **several owners**. - The hierarchical model is a **subset** of the network model. - It uses **set theory** to provide a tree-like hierarchy.

Three crucial components: 1. **Network schema:** the conceptual organization of the whole database. 2. **Subschema:** the portion of the database visible to application programs. 3. **Database management language:** defines data characteristics and structure. It has a Schema DDL, a Subschema DDL and a DML.

Example (slide): A sales system. - SALESREP owns INVOICE (Commission set). - CUSTOMER owns INVOICE (Sales set) and PAYMENT (Payment set). - PRODUCT and INVOICE both own INV_LINE (Inventory set and Line set).

So INVOICE has two owners (SalesRep and Customer), and INV_LINE has two owners (Product and Invoice). That is the “multiple parents” idea. Every individual set is still 1:M.

Analogy: A child has two parents (mother and father), so this is a network. In a pure tree, a child could have only one.

Advantages: - Conceptual simplicity, like the hierarchical model. - Handles more relationship types (but all are 1:M). - Data access flexibility. - Promotes database integrity. - Data independence. - Conformance to standards.

Disadvantages: - **System complexity.** The slide says it was developed by programmers for programmers rather than for users. - **Lack of structural independence.**

Hierarchical vs Network quick comparison

- Hierarchical: each child has **one** parent.
- Network: each member can have **many** owners.
- Both are pointer-based, complex, and lack structural independence.

Lecture 7: Object-Oriented and Relational Models

Recall

HDBS (Hierarchical) and NWDB (Network).

Object-Oriented Database (OODB / ODBMS)

- Data is stored as **objects**, which are **instances of classes**. Classes and objects together form the object-oriented data model.
- A class has **variables** (data) and **messages** (methods or functions).

Example (slide): class CLERK - Variables: name, address, id, salary - Messages: get_name(), get_address(), annual_salary()

Analogy: The class is a cookie cutter. Each object is an actual cookie made from it.

Data objects: the slides show objects written in a JSON-like format. - A city object with _id “37010”, city “ADAMS”, pop 2660, state “TN”, and a nested **councilman** object (name: John Smith, address: 13 Scenic Way). - A book object with title, an **array of authors**, published_date, pages, language, and a nested **publisher** object. - An example showing numbers, strings, null, booleans, dates, an array of objects and an array of integers.

Key idea: objects can **nest** other objects and **contain arrays**. Everything about one thing lives together in one object.

Advantages: - Real-world modeling. - Model simplicity. - Reduced data size.

Disadvantages (the slide heading repeats "Advantages", but these are drawbacks): - Lack of SQL query language. - Lack of automatic constraints (the slide's "automotive" is a typo).

Relational Database Model (RDBM)

- Lets the user or designer operate in a **human logical environment**.
- The user sees the database as a **collection of tables**, while the RDBMS handles physical details.
- **Tables are a series of row/column intersections.**
- Tables are related by sharing **common entity characteristics** (a common column).
- It allows **1:1, 1:M and M:N** relationships (a big step up over the earlier models).

Example (slide): Table AGENT (AGENT_CODE, name, phone) and table CUSTOMER (CUS_CODE, name, phone, renew date, AGENT_CODE). They are **linked through AGENT_CODE**. Customer 10010 has agent 502, so you can look up agent 502 in the AGENT table. In the relational schema, one AGENT can have many (∞) CUSTOMERs.

Analogy: Excel sheets that refer to each other through a shared ID column.

Advantages: - **Structural independence:** the data access path is irrelevant to design, so changing structure does not affect the database. - Improved conceptual simplicity. - Easier design, implementation, management and use. - **Ad hoc query capability with SQL** (a 4GL). - Powerful DBMS.

Disadvantages: - Substantial hardware and system software overhead. - Poor design and implementation are made easy (because it is so easy to use). - May promote **"islands of information"** problems (separate disconnected databases).

Data models classification (slide)

- **Object-based:** Entity-relationship, Semantic, Functional, Object-oriented. These are **knowledge-based**.
- **Record-based (transactions):** Relational, Network, Hierarchical. These are **transaction-based**.

- **Physical:** Unifying, Frame memory. These describe **how data are stored**.
- **Object-relational** bridges the object-based and record-based worlds.

Summary of record-based models

- **Relational** (Oracle, DB2, Sybase, Informix, SQL Server, Ingres): based on mathematical relations, with tables, rows and columns.
- **Network** (CODASYL: Conference on Data System Languages): many-to-many support (the slide's wording), with record types and data items.
- **Hierarchical** (IMS): segment types and fields. In COBOL the equivalent is files, records and fields.

Lecture 8: Functions of DBMS, DBMS Architecture and Metadata

Functions of a DBMS (10 services)

1. **Data storage, retrieval and update.**
2. **User-accessible catalog** (metadata).
3. **Transaction support:** a transaction is a group of operations treated as one unit (all or nothing). Example: transferring money means debit and credit both happen, or neither does.
4. **Concurrency control services:** manage simultaneous users.
5. **Recovery services:** restore data after failure.
6. **Authorization services:** security and permissions.
7. **Support for data communications:** works over networks.
8. **Integrity services:** enforce rules and constraints.
9. **Services to promote data independence.**
10. **Utility services:** backup, import/export, monitoring.

Memory: S-C-T-C-R-A-D-I-D-U (Storage, Catalog, Transaction, Concurrency, Recovery, Authorization, Data communications, Integrity, Data independence, Utility).

DBMS architecture (slide diagram)

- **Programmers** supply **application programs**, which go to the **DML Processor** and become **Program Object Code**.
- **Users** supply **queries**, which go to the **Query Processor**.
- The **DBA** supplies the **database schema**, which goes to the **DDL Processor** and then the **Dictionary Manager**.
- All feed into the **Database Manager**, which is part of the DBMS.
- Below the DBMS, the **Operating System** provides the **File Manager, Access Methods and System Buffers**, which reach the physical storage.

Memory: Programmers use the DML Processor, Users use the Query Processor, the DBA uses the DDL Processor.

Database manager (internal components)

- **Authorization Control:** checks user authorization.
- **Command Processor:** processes the query.
- **Integrity Checker:** checks integrity constraints.
- **Query Optimizer:** determines the optimal (best) strategy.
- **Transaction Manager:** performs the command operation.
- **Scheduler:** manages concurrent operations.
- **Buffer Manager:** transfers data between primary (RAM) and secondary storage (disk).
- **Recovery Manager:** ensures recovery in case of failure.
- The Buffer Manager and Recovery Manager together form the **Data Manager.**

Flow of a request (slide): Query, then Transaction, then Journal, then Buffered, then OS.

Analogy (airport): Security check is Authorization Control. The check-in desk is the Command Processor. A rule checker for passport validity is the Integrity Checker. The route planner is the Query Optimizer. The air-traffic controller is the Scheduler. The emergency team is the Recovery Manager.

Metadata (data dictionary)

- **Metadata is data about data.** It describes the data: names, types, sizes, constraints.

- It lives in the **data dictionary / system catalog**, which sits beside the user databases inside the DBMS. Even a SELECT * FROM tables query on the catalog reads this dictionary.

Types of metadata: 1. **Technical:** system names, table names, table size, data types, values, attributes, and constraints such as foreign key, primary key and indexes. 2. **Business:** ownership of data, policy changes, business rules and regulations. 3. **Operational:** the slide repeats the same description as business metadata (ownership, policies, rules). In general, operational metadata is about how data is processed and used, such as job runs and data lineage. 4. **Descriptive:** describes a file, folder, book, image or video (title, author, date, size, published on).

System catalog stores: - Names of users authorized to access the DBMS. - Names of all data items, their types and sizes, and constraints. - Data items and authorization.

Metadata example (slide): An employee table. - **Data:** the actual rows (44, Simon, Martinez, …). - **Metadata:** the description of columns. employee_id is an int and the primary key. first_name is nvarchar(50). nin is nvarchar(15), the National ID. gender is char(1) with M, F or NULL for unknown. employment_end_date is NULL if the employee is still employed.

Analogy: In a library, the books are the data. The catalog card (title, author, shelf number, ISBN) is the metadata.

Lecture 9: Relational Data Model, Tables, Keys and Constraints

Recall

DBMS functions, architecture, and metadata (data dictionaries).

Objectives of the relational model

- A degree of **data independence.**
- Address data **semantics, consistency and redundancy** problems.
- A **set-oriented** data manipulation language (SQL).

The slide’s picture: a database has data sets selected by criteria, and a presentation method turns the data set into information.

Relational terminology (very examinable)

Formal term	Everyday term
Relation	Table
Tuple	Row (record)
Attribute	Column (field)
Domain	Allowed values for a column
Degree	Number of columns (attributes)
Cardinality	Number of rows (tuples)

- **Relation:** a table that describes an **entity**.
- **Degree of a relation** = number of attributes.
- **Cardinality** = number of tuples.
- **Domain** = the set of all values an attribute can take. Example: the domain of Gender is {M, F}. The domain of Age is whole numbers from 0 to 120.

Memory: Degree = **De**scribes columns (attributes). **Car**dinality = **Car**s in rows (count of rows).

Properties of relations (7 rules)

1. Each relation has a **distinct name.**
2. Each cell contains **exactly one atomic (single) value.** There are **no repeating groups.**
3. Each attribute has a **distinct name.**
4. Values of an attribute come from the **same domain.**
5. The **order of attributes** has no significance.
6. Each tuple is **distinct**, so there are no duplicate rows.
7. The **order of tuples** has no significance.

Example violating rule 2: A Student table with a “Phone” cell holding “0300-111, 0321-222”. That is two values in one cell. Fix it with a separate Phone table or separate columns.

Keys: how to uniquely identify a tuple

The hierarchy goes from broad to narrow:

Super key, then Candidate key, then Primary key.

1. **Super key:** any attribute or set of attributes that uniquely identifies a tuple. It may contain extra, unnecessary attributes.
2. **Candidate key:** a super key with **no unnecessary attribute** (no proper subset is itself a super key). It has two properties:
   - **Uniqueness:** it uniquely identifies the tuple.
   - **Irreducibility:** no subset of it is unique.
   - In SQL, this is the **UNIQUE** constraint.
3. **Primary key (PK):** the candidate key **chosen** to identify tuples. Choose it because it is the most efficient identifier and because it should **remain constant for the life of the tuple.**
   - Types: **single-value PK**, **composite PK**, **surrogate key.**
4. **Foreign key (FK):** an attribute in one table that refers to the primary key of another table. This links tables, for example AGENT_CODE in CUSTOMER pointing to AGENT_CODE in AGENT. In SQL it is an FK constraint.

Analogy: In a class, many things could identify a student: roll number, CNIC, email, and "roll number + name". Each of these is a **super key** (including the last, which has extra baggage). The ones that are minimal (roll number, CNIC, email) are **candidate keys.** The university picks **roll number** as the **primary key.** An exam table that stores a student's roll number is then using it as a **foreign key.**

Worked example from the slide

Relation R(A, B, C, D, E), with A determines BCDE and BC determines ADE. - **Candidate keys:** A and BC (both minimal). - **Primary key:** A (chosen). - **Super keys:** A, BC, ABC, AD, and any other set containing a candidate key.

Rule of thumb: **any set that contains a candidate key is a super key.**

SQL for keys (from the slides)

Unique (candidate) key:

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
```

```
  CONSTRAINT UC_Person UNIQUE (ID, LastName)
);
```

Primary key (a composite one here):

```
CREATE TABLE Persons (
  ID int NOT NULL,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  CONSTRAINT PK_Person PRIMARY KEY (ID, LastName)
);
```

Composite primary key

A primary key made of **more than one attribute** (BC above, or ID plus LastName).

Surrogate key

- An **artificial primary key** generated by the system.
- Numeric and **auto-incremented** for each new row.
- It is a column declared as the PK **instead of a "real" (natural) key.**

When to use it: - To avoid a composite PK. - When the natural PK is used inappropriately or is unstable. - Mostly used in **denormalized** databases.

Analogy: Instead of identifying patients by "name + date of birth + father's name", a hospital simply issues a **patient number.** That number is a surrogate key.

MySQL syntax:

```
CREATE TABLE Persons (
  Personid int NOT NULL AUTO_INCREMENT,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int,
  PRIMARY KEY (Personid)
);
```

SQL Server syntax:

```
CREATE TABLE Persons (
  Personid int IDENTITY(1,1) PRIMARY KEY,
  LastName varchar(255) NOT NULL,
  FirstName varchar(255),
  Age int
);
```

In SQL Server, IDENTITY(1,1) means start at 1 and increase by 1.

Surrogate key advantages: - Reduces composite key complexity. - Flexible retrieval in denormalized databases. - Useful for managing multiple versions of data.

Surrogate key disadvantages: - The value has **no business meaning.** - Users can add **duplicate records**, because the natural data is no longer forced to be unique (unless you also add a UNIQUE constraint).

Self-assignment hints (from the slide)

1. **How do you change the primary key value of an entity?** Update the value in the parent row. If other tables reference it through foreign keys, those references must change too. The clean way is to define the FK with `ON UPDATE CASCADE`, so the change flows to child rows automatically. This is why a PK should rarely change.
2. **How do you change the primary key of a table?** Drop the old PK constraint (and any FKs that depend on it), make sure the new column is NOT NULL and unique, then add the new constraint. In SQL Server:

```
ALTER TABLE Persons DROP CONSTRAINT PK_Person;
ALTER TABLE Persons ADD CONSTRAINT PK_Person PRIMARY KEY (NewColumn);
```

Next lecture preview

SQL constraints, creating subschemas using **views**, and **indexes.**

Final Revision Sheet

Core definitions - Database: shared, logically related data plus its description, built for an organization’s information needs. - DBMS: software to define, create, maintain and control access to the database. - Metadata: data about data (stored in the system catalog or data dictionary).

File system vs DBMS: file systems suffer from redundancy, inconsistency, data dependence, isolation, incompatible formats and fixed queries. A DBMS fixes these with a central repository, but costs more, is more complex and has a higher impact of failure.

DFD: Context, then Level 0, then Level 1 to n. Entity = noun, Flow = data name, Process = verb, Store = noun.

Three-level architecture: External (user view), Conceptual (what), Internal (how). - Logical independence: external schemas are immune to conceptual changes. - Physical independence: the conceptual schema is immune to internal changes.

DDL vs DML: DDL defines structure, and DML manipulates data.

Model timeline: Flat file, Hierarchical, Network, Relational, Object-oriented and Object-relational, then NoSQL.

Model comparison

Model	Structure	Relationship	Key weakness
Hierarchical	Tree, one parent	1:M only	No structural independence, complex pointers
Network	Sets with owner and member, many parents	Multiple owners, each set 1:M	Complex, no structural independence
Object-oriented	Objects of classes (nested)	Contained objects	No SQL, few automatic constraints
Relational	Tables, rows, columns	1:1, 1:M, M:N	Hardware overhead, “islands of information”

Relational vocabulary: Relation = table, Tuple = row, Attribute = column, Degree = number of columns, Cardinality = number of rows, Domain = allowed values.

Keys: Super key, then Candidate key (minimal), then Primary key (chosen). A Foreign key links tables. A Surrogate key is an auto-generated artificial PK.

DBMS database manager parts: Authorization Control, Command Processor, Integrity Checker, Query Optimizer, Transaction Manager, Scheduler, Buffer Manager, Recovery Manager.

FIN !